

Listado de impacto

Componentes del Sistema

1. Base de Datos (PostgreSQL)

Tablas principales:

- **Polizas:** Almacena registros de movimientos de inventario
- **Empleados:** Gestión de empleados autorizados
- **Inventario:** Control de productos y existencias

Polizas:

- id_poliza serial (**PK**)
- empleado_genero int not null (**FK**)
- inventario_sku varchar(50) not null (**FK**)
- cantidad int not null check (Cantidad > 0)
- activo boolean default true
- fecha timestamp default current_timestamp
- created_at timestamp default current_timestamp

Empleados:

- id_empleado serial (**PK**)
- nombre varchar(100) not null
- apellido varchar(100) not null
- puesto varchar(50) not null
- created_at timestamp default current_timestamp

Inventario:

- sku varchar(50) (**PK**)
- nombre varchar(50) not null
- cantidad int not null

- created_at timestamp default current_timestamp

Relaciones:

- Poliza(empleado_genero) -> Empleado(id_empleado) (Many-to-One)
- Poliza(inventario_sku) -> Inventario(sku) (Many-to-One)

Índices optimizados para consultas frecuentes:

- CREATE INDEX idx_empleado_genero ON polizas(empleado_genero);
- CREATE INDEX idx_sku ON polizas(sku);
- CREATE INDEX idx_inventario_created_at ON inventario(created_at);
- CREATE INDEX idx_empleado_created_at ON empleados(created_at);
- CREATE INDEX idx_polizas_created_at ON polizas(created_at);
- CREATE INDEX idx_poliza_estado ON polizas(activo);

2. API REST (Backend) **Spring Boot**

- **Controladores REST:** Los controladores del proyecto se encargan de manejar las solicitudes HTTP y devolver respuestas adecuadas. Cada controlador está anotado con **@RestController** y **@RequestMapping**, lo que indica que es un controlador REST y que se encarga de manejar las solicitudes relacionadas con una entidad específica.

- **PolizaController:** Gestión de operaciones CRUD para pólizas

■ Métodos:

- obtenerPolizas()
 - Tipo: **@GetMapping**
- obtenerPolizaPorId(@PathVariable Long id)
 - Tipo: **@GetMapping("/{id}")**
- crearPoliza(@Valid @RequestBody PolizaDTO polizaDTO)
 - Tipo: **@PostMapping**

- eliminarPoliza(@PathVariable @Min(value = 1, message = "El id debe ser mayor a 0") Long id)
 - Tipo: `@DeleteMapping("/{id}")`
 - actualizarEmpleadoEnPoliza(@PathVariable Long id, @RequestParam Long idEmpleado)
 - Tipo: `@PutMapping("/{id}/empleado")`
- **EmpleadoController:** Administración de empleados.
 - **Metodos:**
 - obtenerEmpleados()
 - Tipo: `@GetMapping`
 - obtenerEmpleadoPorId(@PathVariable Long id)
 - Tipo: `GetMapping("/{id}")`
- **InventarioController:** Control de inventario y existencias
 - **Métodos:**
 - obtenerArticulos()
 - Tipo: `@GetMapping`
 - obtenerArticuloPorSku(@PathVariable String sku)
 - Tipo: `GetMapping("/{sku}")`
- **Servicios de negocio:** Son las clases que contienen la lógica de negocio de la aplicación. En este caso, hay tres servicios: **PolizaService**, **EmpleadoService** e **InventarioService**. Cada servicio se encarga de realizar las operaciones necesarias para cumplir con los requisitos de la aplicación.
 - **PolizaService:** Lógica de negocio para pólizas
 - **Métodos:**
 - obtenerPolizas()
 - obtenerPolizaPorId(Long id)
 - crearPoliza(Long idEmpleado, String sku, int cantidad)
 - eliminarPoliza(Long idPoliza)
 - actualizarEmpleadoEnPoliza(Long idPoliza, Long idEmpleado)

- **EmpleadoService:** Gestión de empleados
 - **Metodos:**
 - obtenerEmpleados()
 - obtenerEmpleadoPorId(Long id)
- **InventarioService:** Control de inventario
 - **Métodos:**
 - obtenerArticulos()
 - obtenerArticuloPorSku(String sku)
- **Entity:** Las entidades del proyecto representan las tablas de la base de datos y tienen atributos y relaciones que se utilizan para almacenar y recuperar datos.
 - **Poliza:** Esta entidad representa un registro de la tabla póliza
 - **Atributos:**
 - **id:** el ID de la póliza.
 - **empleadoGenero:** el ID del empleado que generó la póliza.
 - **inventarioSku:** el SKU del inventario asociado a la póliza.
 - **cantidad:** la cantidad de productos de la poliza.
 - **activo:** un boolean que indica si la póliza está activa o no.
 - **created_at:** la fecha en que se creó la póliza.
 - **Relaciones**
 - **Empleado:** la póliza está asociada a un empleado.
 - **Inventario:** la póliza está asociada a un inventario.
 - **Empleado:** Esta entidad representa un empleado.
 - **Atributos:**
 - **id:** el ID del empleado.
 - **nombre:** el nombre del empleado.
 - **apellido:** el apellido del empleado.
 - **puesto:** el puesto del empleado.
 - **created_at:** la fecha en que se creó el empleado.
 - **Relaciones**

- **Poliza:** el empleado está asociado a varias pólizas.
- **Inventario:** Esta entidad representa un inventario.
 - **Atributos:**
 - **sku:** el SKU del inventario.
 - **nombre:** el nombre del inventario.
 - **cantidad:** la cantidad de productos en el inventario.
 - **created_at:** la fecha en que se creó el inventario.
 - **Relaciones:**
 - **Poliza:** el inventario está asociado a varias pólizas.
- **Repositorios:** Los repositorios del proyecto se encargan de interactuar con la base de datos y realizar operaciones CRUD en las entidades. Cada repositorio está anotado con **@Repository** y **@Transactional**, lo que indica que es un repositorio y que las operaciones que se realizan en él son transaccionales.
 - **PolizaRepository:** Este repositorio se encarga de interactuar con la tabla o entity **Polizas** de la base de datos.
 - **Métodos:**
 - **findAll():** devuelve una lista de todas las pólizas.
 - **findById(Long id):** devuelve una póliza por su ID.
 - **save(Poliza poliza):** crea o actualiza una póliza.
 - **deleteById(Long id):** elimina una póliza por su ID.
 - **EmpleadoRepository:** Este repositorio se encarga de interactuar con la tabla **Empleados** de la base de datos.
 - **Métodos:**
 - **findAll():** devuelve una lista de todos los empleados.
 - **findById(Long id):** devuelve el empleado por su ID.

- **InventarioRepository:** Este repositorio se encarga de interactuar con la tabla **Inventario** de la base de datos.
 - **Métodos:**
 - **findBySku(String sku):** devuelve un artículo por su sku.
- **DTOs:** Los DTOs (Data Transfer Objects) que se utilizan para transferir datos entre las capas de la aplicación sin comprometer datos sensibles de los entities. En el proyecto, los DTOs se utilizarán para transferir datos entre las capas de la aplicación, como por ejemplo, entre la capa de presentación y la capa de negocio.
- **PolizaDTO:** Este DTO se utiliza para transferir datos de una póliza entre las capas de la aplicación.
 - **Atributos:**
 - **id:** el ID de la póliza.
 - **empleadoGenero:** el ID del empleado que generó la póliza.
 - **inventarioSku:** el SKU del inventario asociado a la póliza.
 - **cantidad:** la cantidad de productos asegurados.
 - **activo:** un boolean que indica si la póliza está activa o no.
 - **fecha:** la fecha en que se creó la póliza.
- **EmpleadoDTO:** Este DTO se utiliza para transferir datos de un empleado entre las capas de la aplicación.
 - **Atributos:**
 - **id:** el ID del empleado.
 - **nombre:** el nombre del empleado.
 - **apellido:** el apellido del empleado.
 - **puesto:** el puesto del empleado.
- **InventarioDTO:** Este DTO se utiliza para transferir datos de un inventario entre las capas de la aplicación.
 - **Atributos:**
 - **sku:** el SKU del inventario.
 - **nombre:** el nombre del inventario.

- **cantidad:** la cantidad de productos en el inventario.
- **PolizaResponseDTO:** Este DTO se utiliza para presentar una respuesta de póliza personalizada.
 - **Atributos:**
 - **PolizaResDTO:** Es un DTO anidado que contiene.
 - **idPoliza:** Id de la póliza.
 - **cantidad:** Cantidad de artículos.
 - **fecha:** fecha de creación de la póliza.
 - **EmpleadoPolizaResDTO:** Es un DTO anidado que contiene.
 - **idEmpleado:** ID del empleado.
 - **nombre:** Nombre del empleado.
 - **apellido:** Apellido del empleado
 - **InventarioPolizaResDTO:** Es un DTO anidado que contiene.
 - sku: Sku del artículo en la póliza.
 - nombre: Nombre del artículo en la póliza.
- **ApiResponseDTO:** Este DTO se utiliza como envoltorio de respuesta HTTP genérica para mantener un congruencia con el formato de respuesta .
 - **Atributos:**
 - **message:** Mensaje del resultado de la petición.
 - **data:** Representa el cuerpo de la respuesta de la petición.
 - **codeHttp:** Devuelve el código numérico de la respuesta de la petición
 - **descriptionHttp:** Devuelve la descripción del código de respuesta de la petición
 - **errors:** Devuelve la descripción o lista de mensaje de errores
 - **timestamp:** Devuelve la fecha y hora de la petición
- **Validaciones con Jakarta Validation:** Validaciones con Jakarta Validation: Se utiliza para validar los datos que se ingresan en la aplicación. En este caso, se

han definido varias anotaciones para validar los campos de los DTOs, como @NotNull, @NotBlank, @Min, @Size, @Positive y @NotBlank.

- **@NotNull, @NotBlank** para campos requeridos
 - **@Min, @Size** para validaciones de rango
 - **@Positive** para valores numéricos
 - **@NotBlank** para valores vacíos
- **Manejo de excepciones personalizado:** Se utiliza para manejar las excepciones que se producen en la aplicación de manera centralizada. En este caso, se han definido dos excepciones personalizadas:
ResourceNotFoundException, DuplicateResourceException y GlobalExceptionHandler.
 - **ResourceNotFoundException:** Cuando el elemento a buscar no es encontrado.
 - **DuplicateResourceException:** Cuando el elemento ya existe y hay una restricción de duplicidad.
 - **GlobalExceptionHandler** para control centralizado de excepciones y administración de estas, retornando una estructura de respuesta genérica con la información del problema tratada para retornar como respuesta http.
- **Documentación con OpenAPI 3.0:** Documentación con OpenAPI 3.0: Se utiliza para generar documentación para la API REST. En este caso, se ha configurado Swagger UI para que se pueda acceder a la documentación en `/api/swagger-ui.html`.
- **Capa de persistencia con JPA/Hibernate:** Se utiliza para interactuar con la base de datos. En este caso, se ha configurado JPA/Hibernate para que se pueda acceder a la base de datos de manera eficiente mediante las interfaces de repository.

- **Logger con Logback:** El archivo logback.xml es un archivo de configuración para el logger Logback, que es un framework de logging para Java.
 - **Ubicación:** El archivo logback.xml se encuentra en el directorio src/main/resources del proyecto.
 - **Contenido:** El archivo logback.xml contiene la configuración para el logger Logback, incluyendo la definición de los niveles de logging, los formatos de los mensajes de log y los destinos de los logs.
 - **Configuración del logger**
 - **Root logger:** El logger raíz se configura con un nivel de logging de INFO y se envían los mensajes de log a un archivo llamado application.log.
 - **Appender:** Se define un appender llamado FILE que envía los mensajes de log a un archivo llamado application.log.
 - **PatternLayout:** Se define un patrón de formato para los mensajes de log que incluye la fecha y hora del evento, el nivel de logging, el nombre de la clase y el mensaje de log.
- **Test unitarios:**
 - **Dependencias**
 - JUnit: 4.12
 - Mockito: 2.28.2
 - **Tecnologías**
 - **JUnit:** Framework de testing para Java.
 - **Mockito:** Biblioteca de mocking para Java.
 - **Clases: PolizaServiceTest**
 - **Métodos:**
 - testObtenerPolizas()
 - testObtenerPolizaPorId_Existente()
 - testObtenerPolizaPorId_NoExistente()
 - testCrearPolizaExito()
 - testCrearPolizaFalloStockInsuficiente()
 - testEliminarPolizaExito()
 - testEliminarPolizaNoEncontrada()
 - testActualizarEmpleadoEnPolizaExito()

- `testActualizarEmpleadoEnPolizaNoEncontrada()`

- **Características Técnicas**

- Clasificación de la arquitectura:
 - Arquitectura de 3 capas
 - Separa la UI, la lógica de negocio y la base de datos.
 - Escalable y fácil de mantener.
 - Arquitectura Cliente Servidor
 - El cliente (React) consume servicios desde un servidor API (Spring Boot).
 - RESTful
- Patrones de Diseño:
 - DTO Pattern
 - Repository Pattern
 - Builder Pattern (ApiResponseBuilder)
 - Inyección de dependencias

- **Control de versiones:** Git y github

- **Gestión de dependencias:**

- **Maven** con pom.xml estructurado

3. Web Application (Frontend) **React**

- **PolizasList** : Es la vista principal para la gestión de pólizas. Este componente es el corazón de la aplicación, donde los usuarios pueden ver y gestionar todas las pólizas del sistema de manera eficiente y amigable. Sus funciones principales son:
 - 1. **Visualización de Datos**
 - - Muestra una tabla interactiva con todas las pólizas
 - - Implementa paginación para grandes conjuntos de datos

- - Presenta información detallada de cada póliza
 - - Permite ordenamiento por columnas
- 2. **Funcionalidades CRUD**
 - - Botón para crear nuevas pólizas
 - - Opciones de edición para cada póliza
 - - Función de eliminación con confirmación
 - - Actualización en tiempo real
- 3. **Características Avanzadas**
 - - Sistema de búsqueda y filtrado
 - - Ordenamiento personalizado
 - - Feedback visual de acciones
 - - Animaciones de transición
 - - Manejo de estados de carga
- 4. **Integración**
 - - Conecta con los servicios API
 - - Maneja estados globales
 - - Implementa navegación programática
 - - Gestiona errores y excepciones.
- 5. **Endpoints que consulta**
 - Para alimentar la tabla de pólizas.
 - GET /api/polizas
 - Para eliminar pólizas que ya no se necesiten.
 - DELETE /api/polizas/{id}
 - Para aplicar búsquedas en el panel de pólizas.
 - GET /api/polizas?search={término}
 - Para aplicar el ordenamiento de las pólizas.
 - GET /api/polizas?sort={campo}
- **PolizaForm** : Es el formulario para crear nuevas pólizas. Este componente proporciona una interfaz intuitiva y robusta para la creación de nuevas pólizas, asegurando la validación de datos y una experiencia de usuario fluida. Sus principales características son:
 - 1. **Gestión de Formulario**

- - Campos validados para datos de póliza
 - - Selección de empleados mediante Autocomplete
 - - Manejo de estado del formulario
 - - Validaciones en tiempo real
- **2. Interacción con Usuario**
 - - Feedback visual inmediato
 - - Mensajes de error específicos
 - - Indicadores de carga
 - - Notificaciones de éxito/error
 - - Confirmaciones de acciones
- **3. Integración de Datos**
 - - Conexión con servicios API
 - - Carga de catálogos de empleados
 - - Manejo de respuestas del servidor
 - - Actualización del estado global
- **4. Características UI**
 - - Layout responsivo con Grid
 - - Componentes Material-UI
 - - Animaciones de transición
 - - Botones de acción (guardar/cancelar)
 - - Manejo de estados de carga
- **5. Endpoints que consulta:**
 - Para mandar a crear una nueva póliza
 - POST /api/polizas
 - Para obtener listado de empleados y alimentar el input con las opciones.
 - GET /api/empleados
 - Para obtener el listado de artículos y alimentar el input con las opciones.
 - GET /api/inventario
- **PolizaEditForm** : El componente está diseñado para la edición de pólizas existentes. Este componente asegura una experiencia fluida en la modificación

de pólizas existentes, manteniendo la integridad de los datos y proporcionando feedback claro al usuario. Sus principales características son:

- **1. Carga y Manejo de Datos**
 - - Obtiene datos de la póliza existente usando el ID
 - - Precarga la información en el formulario
 - - Maneja estados de carga inicial
 - - Gestiona la actualización de datos
- **2. Funcionalidades de Edición**
 - - Modificación de datos de la póliza
 - - Cambio de empleado asignado
 - - Actualización de inventario
 - - Validaciones específicas para edición
 - - Control de cambios realizados
- **3. Interfaz de Usuario**
 - - Formulario pre-poblado
 - - Indicadores de carga
 - - Mensajes de confirmación
 - - Alertas de error
 - - Navegación post-edición
- **4. Integración**
 - - Usa useParams para obtener el ID
 - - Conecta con múltiples endpoints API
 - - Maneja estados de actualización
 - - Gestiona respuestas del servidor
 - - Redirección tras éxito
- **5. Endpoints que consulta:**
 - Para obtener la póliza elegida es usada para llenar los datos del input (Cantidad).
 - GET /api/polizas/{id}
 - Para mandar a actualizar la póliza editada.
 - PUT /api/polizas/{id}
 - Para actualizar el empleado de la póliza en caso de error de dato
 - PUT
/api/polizas/{id}/empleado?idEmpleado={idEmpleado}

- Para obtener el listado de empleados y mostrarlo en el input
 - GET /api/empleados
 - Para obtener el listado de artículos de inventario y llenar el input de artículo con todos sus datos.
 - GET /api/inventario
- **MainLayout:** Es el componente que define la estructura principal de la aplicación. Este componente actúa como el "esqueleto" de la aplicación, proporcionando una estructura coherente y profesional para todas las páginas. Sus funciones principales son:
 - **1. Estructura Base**
 - - Define el layout principal de la aplicación
 - - Implementa la barra de navegación superior (AppBar)
 - - Incluye el logo de Coppel y el título del sistema
 - - Maneja el contenedor principal donde se renderizan las páginas
 - **2. Componentes Material-UI**
 - - Utiliza AppBar para la barra superior
 - - Implementa Toolbar para la organización de elementos
 - - Usa Box para el contenedor principal
 - - Maneja estilos y espaciado consistente
 - **3. Gestión de Contenido**
 - - Utiliza Outlet de React Router para renderizar las rutas hijas
 - - Mantiene una estructura consistente en toda la aplicación
 - - Implementa animaciones con Framer Motion
 - - Gestiona la disposición responsiva
 - **4. Características**
 - - Diseño responsivo
 - - Animaciones fluidas entre cambios de página
 - - Estructura consistente para todas las vistas
 - - Manejo de estilos centralizados
- **Routes:** Es fundamental para la navegación de la aplicación. Es esencialmente el "mapa de navegación" de la aplicación, controlando qué componente se debe mostrar según la URL actual. Sus principales funciones son:

- **1. Gestión de Rutas**
 - - Define todas las URLs disponibles en la aplicación
 - - Conecta cada ruta con su componente correspondiente
 - - Maneja la navegación entre páginas de pólizas
 - **2. Estructura de Navegación**
 - - Implementa el layout principal
 - - Organiza la jerarquía de páginas
 - - Gestiona las rutas para:
 - - Lista de pólizas
 - - Creación de pólizas
 - - Edición de pólizas
 - **3. Control de Acceso**
 - - Maneja la estructura de navegación
 - - Asegura que los componentes correctos se muestren en cada ruta
 - - Facilita la transición entre diferentes vistas
- **api.js:** Es crucial para la comunicación con el backend. Este archivo actúa como una capa de abstracción entre el frontend y el backend, centralizando toda la lógica de comunicación HTTP y facilitando el mantenimiento y la consistencia en las llamadas a la API. Sus principales funciones son:
 - **1. Configuración de Axios**
 - - Establece la configuración base para todas las peticiones HTTP
 - - Define la URL base para las APIs
 - - Configura los headers comunes
 - **2. Endpoints de Pólizas**
 - - Métodos CRUD completos:
 - - Obtener todas las pólizas
 - - Crear nueva póliza
 - - Actualizar póliza existente
 - - Eliminar póliza
 - - Asignar empleado a póliza
 - **3. Manejo de Respuestas**
 - - Procesamiento de respuestas del servidor

- - Transformación de datos
- - Manejo centralizado de errores
- **4. Interceptores**
 - - Modifica requests antes de enviarlos
 - - Procesa responses antes de entregarlos
 - - Maneja errores de forma global